# Acceso a SQLite desde Python

### Conexión · Cursor · Comandos · Resultados · Transacciones

Esta guía presenta los elementos básicos que utiliza Python para trabajar con una base de datos SQLite. El objetivo es comprender qué función cumple cada objeto antes de implementar operaciones de Alta, Baja, Modificación y Consulta (ABMC).

## 1. ¿Cómo se comunica Python con SQLite?

Python incluye el módulo sqlite3. No es necesario instalar un servidor de base de datos: SQLite almacena la información en un archivo y el programa accede a él directamente.

| Programa Python | Conexión | Cursor | Comando SQL | Base SQLite |
|---|---|---|---|---|

El recorrido conceptual es: Programa → Conexión → Cursor → SQL → SQLite.

## 2. La conexión: sqlite3.Connection

La conexión representa el vínculo entre el programa Python y una base de datos. Se obtiene con sqlite3.connect(). Si el archivo no existe, SQLite lo crea.

```
import sqlite3

conexion = sqlite3.connect("escuela.db")
```

A través de la conexión se pueden realizar, entre otras, estas operaciones:

| Operación | Función |
|---|---|
| conexion.cursor() | Crear un cursor. |
| conexion.commit() | Confirmar los cambios de una transacción. |
| conexion.rollback() | Deshacer cambios pendientes. |
| conexion.close() | Cerrar la conexión con la base de datos. |

## 3. El cursor: sqlite3.Cursor

El cursor es el objeto que utilizamos para ejecutar sentencias SQL y recuperar sus resultados. Se crea a partir de una conexión.

```
cursor = conexion.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS alumnos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        edad INTEGER NOT NULL
    )
""")
```

**Idea clave:** la conexión administra el vínculo y la transacción; el cursor ejecuta sentencias SQL y permite recuperar resultados.

## 4. Comandos SQL y execute()

El comando SQL que enviamos a SQLite es un string de Python. Las triples comillas permiten escribirlo en varias líneas y facilitan su lectura. No forman parte del lenguaje SQL.

```
comando = """
    INSERT INTO alumnos (nombre, edad)
    VALUES (?, ?)
"""

cursor.execute(comando, ("María", 19))
```

Los signos ? son parámetros. Los valores se entregan por separado y en el mismo orden. Esta forma evita concatenar los datos dentro del SQL y debe utilizarse cuando los valores provienen de variables.

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))
```

```
cursor.execute("""
    INSERT INTO alumnos (nombre, edad)
    VALUES (?, ?)
""", (nombre, edad))
```

## 5. Consultas y recuperación de resultados

Una sentencia SELECT no modifica datos: obtiene filas de la base. Después de ejecutarla, el cursor permite recuperar el resultado. Para comenzar son suficientes fetchone() y fetchall().

```
cursor.execute("""
    SELECT id, nombre, edad
    FROM alumnos
""")

alumnos = cursor.fetchall()

for alumno in alumnos:
    print(alumno)
```

| Método | Resultado |
|---|---|
| fetchone() | Obtiene una fila del resultado; si no hay más, devuelve None. |
| fetchall() | Obtiene todas las filas restantes del resultado. |

Por defecto, cada fila recuperada puede trabajarse como una tupla, por ejemplo: (3, 'María', 19).

## 6. Transacciones: commit() y rollback()

Una transacción agrupa cambios que deben tratarse como una unidad de trabajo. Esto es importante cuando varias operaciones forman parte de una misma acción.

```
cursor.execute("""
    UPDATE cuentas
    SET saldo = saldo - ?
    WHERE id = ?
""", (10000, 1))

cursor.execute("""
    UPDATE cuentas
    SET saldo = saldo + ?
    WHERE id = ?
""", (10000, 2))

conexion.commit()
```

Si las operaciones se completan correctamente, commit() confirma los cambios. Si se produce un problema antes de confirmarlos, rollback() permite deshacer los cambios pendientes de la transacción.

```
try:
    # operaciones que modifican la base
    conexion.commit()
except:
    conexion.rollback()
```

**Importante:** no conviene presentar commit() como una línea que se copia automáticamente. Su significado es confirmar una transacción.

## 7. Relación con las operaciones ABMC

| Operación | SQL | Ejemplo |
|---|---|---|
| Alta | INSERT | Agregar un alumno. |
| Baja | DELETE | Eliminar un alumno. |
| Modificación | UPDATE | Cambiar edad o nombre. |
| Consulta | SELECT | Buscar o listar alumnos. |

## 8. Ejemplo integrado

El siguiente programa crea una base, crea una tabla, agrega un alumno y luego lista los registros:

```
import sqlite3

conexion = sqlite3.connect("escuela.db")
cursor = conexion.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS alumnos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        edad INTEGER NOT NULL
    )
""")

nombre = input("Nombre: ")
edad = int(input("Edad: "))

cursor.execute("""
    INSERT INTO alumnos (nombre, edad)
    VALUES (?, ?)
""", (nombre, edad))

conexion.commit()

cursor.execute("SELECT id, nombre, edad FROM alumnos")
alumnos = cursor.fetchall()

for alumno in alumnos:
    print(alumno)

conexion.close()
```

## 9. Resumen

| Elemento | Responsabilidad principal |
|---|---|
| sqlite3.connect() | Abrir o crear la base de datos. |
| Connection | Mantener la conexión y controlar la transacción. |
| Cursor | Ejecutar SQL y recuperar resultados. |
| execute() | Enviar una sentencia SQL a SQLite. |
| fetchone() / fetchall() | Obtener filas producidas por un SELECT. |
| commit() | Confirmar los cambios. |
| rollback() | Deshacer cambios pendientes. |
| close() | Cerrar la conexión. |